SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Coding Challenge Task

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	**CO4:** Articulation (BL4, BL5)	**Assessment:**	Assessment Tool 1 – Coding Challenge Task
Weightage:	40% of CIA	**Total Marks:**	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	ATHIN SIVA.S	**Reg. No.:**	192524237
Section / Batch:	2025	**Date:**	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Write clear code for the given problem.
- Analyze the Time complexity and Space complexity of your code using dynamic programming and greedy strategies

Question Type	Focus Area	Questions
Coding Challenge Task	Focus on Code and analyze optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Coding Challenge Task

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Coding Challenge Task

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%				
Algorithm	25%				

Code Implementation	20%				
Competitive Performance	20%				
Test Case Handling	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

CO4 AT1-CODE CHALLANGE TASK OPEN ✓ Completed

10/1 answered

Review →

View Rubric

1. Problem Statement

Given n pairs of numbers, where each pair contains a start value and an end value, the objective is to form the longest possible chain of pairs. A pair (a, b) can be followed by another pair (c, d) only when b < c. The task is to determine the maximum number of pairs that can be included in such a chain. This problem can be solved efficiently using a Greedy approach or Dynamic Programming.

2. Approach Used – Greedy Algorithm

We use the Greedy technique by sorting all pairs according to their ending value. After sorting, we repeatedly select the pair whose end value is the smallest among the remaining compatible pairs. This choice leaves maximum room for the remaining pairs and therefore helps obtain the longest chain.

The important condition is that the end value of the previously selected pair must be strictly less than the start value of the current pair. Therefore, if the current pair is (start, end), it is selected only when start > last_end.

3. Algorithm

1. Read the number of pairs n.
2. Read all n pairs as (start, end).
3. Sort the pairs in ascending order based on their end value.
4. Initialize last_end to a sufficiently small value and chain_length to 0.
5. Traverse the sorted pairs one by one.
6. If start > last_end, select the pair and increase chain_length by 1.
7. Update last_end with the selected pair's end value.
8. After processing all pairs, print the maximum chain length.

4. Python Program

```
def max_chain_length(pairs):
    pairs.sort(key=lambda x: x[1])

    last_end = float('-inf')
    chain_length = 0

    for start, end in pairs:
        if start > last_end:
            chain_length += 1
            last_end = end
```

```
    return chain_length


n = int(input())
pairs = []

for _ in range(n):
    start, end = map(int, input().split())
    pairs.append((start, end))

result = max_chain_length(pairs)
print("Max Chain Length =", result)
```

5. Sample Input

3
5 24
15 25
27 40

6. Sample Output

Max Chain Length = 2

7. Explanation of Sample

The given pairs are (5, 24), (15, 25), and (27, 40). After sorting by end value, the order remains the same. We first select (5, 24). The next pair (15, 25) cannot be selected because 15 is not greater than 24. The pair (27, 40) can be selected because 27 > 24. Hence, the longest chain is (5, 24) → (27, 40), containing 2 pairs.

8. Correctness of the Greedy Approach

The greedy choice selects the compatible pair with the smallest ending value. Choosing a pair that finishes earlier cannot reduce the possibility of selecting future pairs, because all later compatible pairs must start after the selected end. Thus, an earlier finishing pair leaves at least as much space for the remaining pairs as any pair finishing later. Repeating this choice produces a maximum-length chain.

9. Time and Space Complexity

Time Complexity: O(n log n), because sorting the n pairs requires O(n log n) time and the subsequent traversal takes O(n) time.

Space Complexity: O(n) in the worst case for storing the input pairs. The greedy selection itself uses only O(1) additional variables apart from the storage used by the list.

10. DAA Concepts Used

Greedy method

Sorting based on a suitable selection criterion

Optimal local choice

Time-complexity analysis

Construction of an optimal chain

11. Conclusion

The Maximum Length Chain of Pairs problem can be solved efficiently using the Greedy method. By sorting pairs according to their ending values and always selecting the next compatible pair with the earliest end, the algorithm obtains the maximum possible chain length. The overall time complexity is O(n log n), making the solution suitable for large inputs.